Memory Systems

Introduction; performance analysis; Caches - basic concepts, Cache mapping, Cache replacement, Multiple-Level Caches, Reducing Miss Rate, Write Policy; Virtual Memory - Address Translation; Page Table; Translation Lookaside Buffer; Memory Protection.

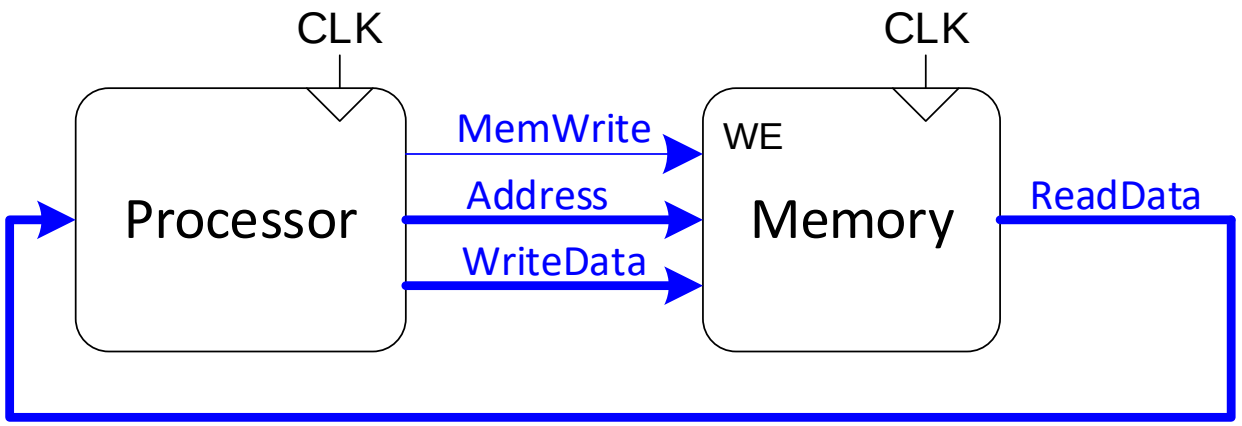
Introduction

Memory System Overview

- The memory system is responsible for storing program instructions, input data, intermediate results, and output data.
- Computer system performance depends not only on processor speed but also on memory system performance and the interface between the processor and memory.

- The processor communicates with the memory system over a *memory interface*.
- Figure 8.1 shows the simple memory interface used in our multicycle RISC-V processor.
- The processor sends an address over the *Address* bus to the memory system.
- For a read, *MemWrite* is 0 and the memory returns the data on the *ReadData* bus.
- For a write, *MemWrite* is 1 and the processor sends data to memory on the *WriteData* bus.

Figure 8.1 The memory interface



Processor-Memory Gap

- In early computer systems, memory access time was close to processor speed.
- However, modern processors operate much faster than memory systems.
- DRAM memory is significantly slower compared to CPU clock speed. This growing speed gap causes the processor to wait frequently for memory, reducing system performance.

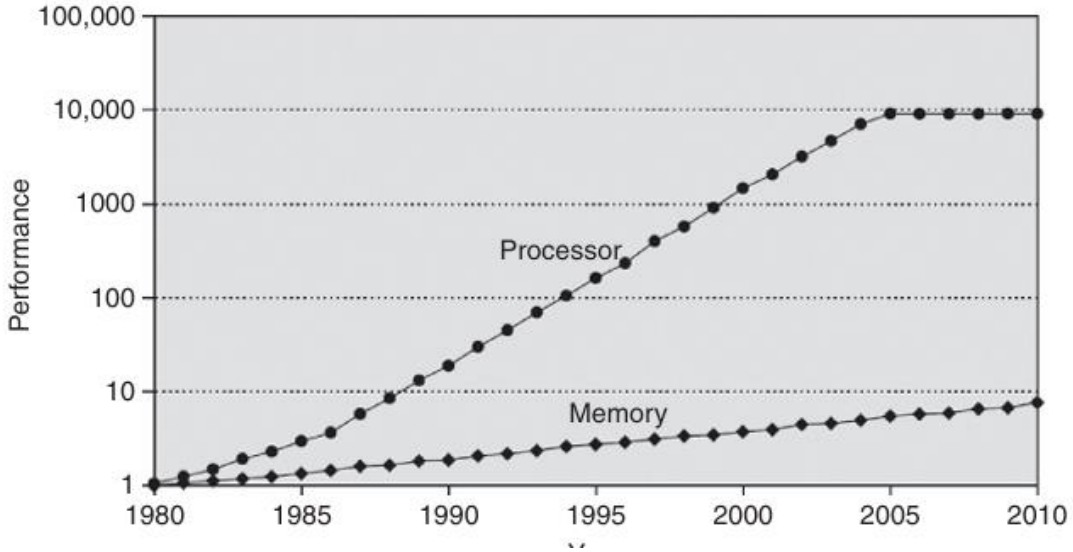


Figure 8.2 Diverging processor and memory performance.

- The major issues in memory system design can be broadly explained using a metaphor of books in a library.
- A library contains many books on the shelves.
- If you were studying on a subject Computer Organization, you might go to the library1 and pull Harris's *Digital Design and Computer Architecture - RISC-V Edition* off the shelf and bring it to your cubicle.
- After referring it, you might put it back and pull out Hamacher's *Computer Organization and Embedded Systems*.
- You might then go back for another reference from *Modern Computer Architecture and Organization*, followed by yet another trip to *Computer Organization and Architecture Designing for Performance*.

Locality of Reference

- Exploit locality to make memory accesses fast
- Use the principles of temporal and spatial locality.
- Temporal locality means that if you have used a book recently, you are likely to use it again soon.
- Spatial locality means that when you use one particular book, you are likely to be interested in other books on the same shelf.
- The library itself makes the common case fast by using these principles of locality.

- obtain the benefits of both a large collection and quick access to the most commonly used books through a hierarchy of storage.
- The most commonly used books are in your cubicle. A larger col lection is on the shelves. And an even larger collection is available, with advanced notice, from the basement and other libraries.
- Similarly, memory systems use a hierarchy of storage to quickly access the most commonly used data while still having the capacity to store large amounts of data.

Locality of Reference

Exploit locality to make memory accesses fast:

- **Temporal Locality:**
  - Locality in time
  - If data used recently, likely to use it again soon
  - **How to exploit:** keep recently accessed data in higher levels of memory hierarchy
- **Spatial Locality:**
  - Locality in space
  - If data used recently, likely to use nearby data soon
  - **How to exploit:** when access data, bring nearby data into higher levels of memory hierarchy too

Memory System Challenge

- The ideal memory should be fast, large in capacity, and inexpensive.
- In practice, it is impossible to achieve all three simultaneously.
- Designers therefore use a memory hierarchy to balance these tradeoffs.
- Speed is characterized by both latency and throughput.
- Memory latency is the time to access the first byte of information.
- Throughput is the number of bytes per second that can be delivered. Many memories have good throughput but long latency.

Memory Hierarchy

The memory hierarchy organizes memory into multiple levels based on speed, cost, and size.

The typical hierarchy includes registers, cache memory, main memory (DRAM), and secondary storage such as SSDs and hard drives.

Frequently accessed data is stored closer to the CPU in faster memory levels.

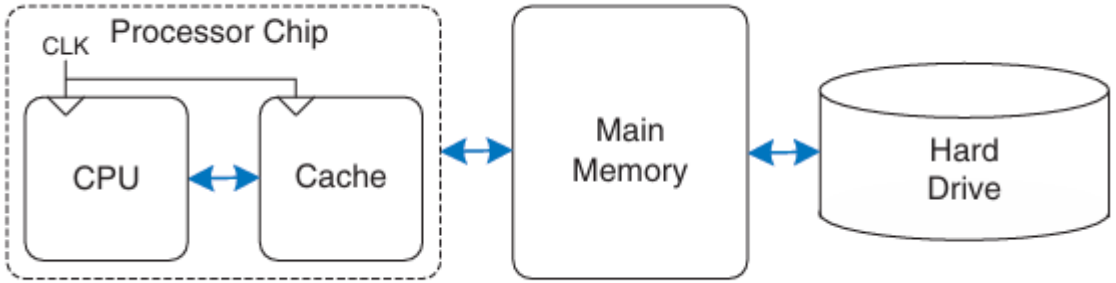
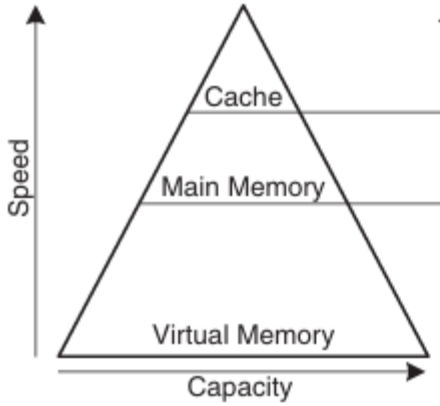


Figure 8.3 A typical memory hierarchy



Technology	Price / GB	Access Time (ns)	Bandwidth (GB/s)
SRAM	$100	0.2–3	100+
DRAM	$3	10–50	30
SSD	$0.10	20,000	0.05–3
HDD	$0.03	5,000,000	0.001–0.1

Figure 8.4 Memory hierarchy components, with typical characteristics in 2021

Memory Interface

- The processor communicates with memory using an address bus to specify memory locations, a data bus to transfer data, and control signals to indicate read or write operations.

Cache Memory

- Cache memory is a small, fast memory placed close to the processor.
- It is built using SRAM technology and stores frequently used instructions and data.
- Cache memory helps reduce average memory access time and improves overall system performance.

MEMORY PERFORMANCE ANALYSIS

Memory system performance is an important factor in overall computer performance.

It determines how quickly the processor can access instructions and data.

Performance is measured using hit rate, miss rate, and average memory access time.

Hit and Miss

When the processor accesses memory, the data may or may not be present in the cache.

Hit: When requested data is found in the cache.

Miss: When requested data is not found in the cache and must be fetched from lower levels of memory.

a. **Hit Rate** : Hit rate is defined as the fraction of memory accesses that are found in the cache.

Formula:

$$Hit\,Rate = \frac{Number\,of\,hits}{Number\,of\,total\,memory\,accesses} = 1 - Miss\,Rate$$

A higher hit rate indicates better cache performance.

b. **Miss Rate**

Miss rate is defined as the fraction of memory accesses that are not found in the cache.

Formula:

$$Miss\ Rate = \frac{Number\ of\ misses}{Number\ of\ total\ memory\ accesses} = 1 - Hit\ Rate$$

A lower miss rate improves system performance.

Average Memory Access Time (AMAT):

is the average time a processor must wait for memory per load or store instruction.

In the typical computer system, the processor first looks for the data in the cache.

If the cache misses, the processor then looks in main memory.

If the main memory misses, the processor accesses virtual memory on the hard disk. Thus, AMAT is calculated as:

$$AMAT = t_{\text{cache}} + MR_{\text{cache}}(t_{MM} + MR_{MM}t_{VM})$$

where t_{cache}, t_{MM}, and t_{VM} are the access times of the cache, main memory, and virtual memory, and MR_{cache} and MR_{MM} are the cache and main memory miss rates, respectively.

Importance of AMAT:

AMAT is used to evaluate memory system efficiency.

A lower AMAT means faster average access time and better system performance.

Q) Suppose a program has 2,000 data access instructions (loads or stores) and 1,250 of these requested data values are found in the cache. The other 750 data values are supplied to the processor by main memory or disk memory. What are the miss and hit rates for the cache?

Solution

The miss rate is 750/2000 = 0.375 = 37.5%.

The hit rate is 1250/2000 = 0.625 = 1 − 0.375 = 62.5%.

Q) Suppose processor has 2 levels of hierarchy: cache and main memory t_{cache} = 1 cycle, t_{MM} = 100 cycles What is the AMAT (average memory access time) of the program from Example 1?

$$\begin{aligned} \text{AMAT} &= t_{\text{cache}} + MR_{\text{cache}}(t_{MM}) \\ &= [1 + 0.375(100)] \text{ cycles} \\ &= 38.5 \text{ cycles} \end{aligned}$$

Q) Suppose a computer system has a memory organization with only two levels of hierarchy, a cache and main memory. What is the average memory access time given the access times and miss rates in Table?

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

Solution

The average memory access time is 1 + 0.1(100) = 11 cycles.

Q) An 11-cycle average memory access time means that the processor spends ten cycles waiting for data for every one cycle actually using that data. What cache miss rate is needed to reduce the average memory access time to 1.5 cycles given the access times in Table 8.1?
Solution

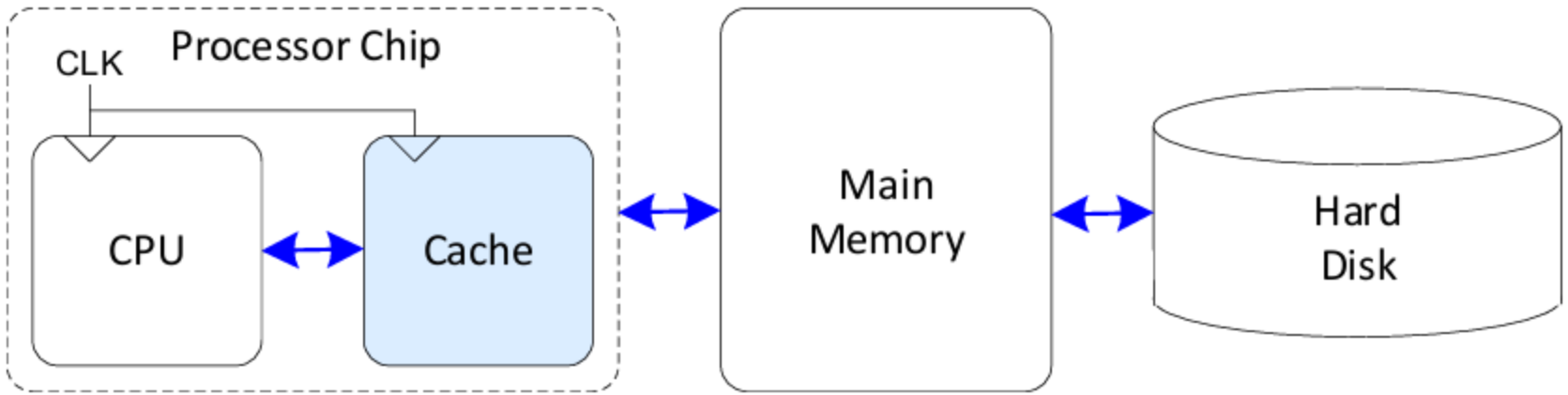
If the miss rate is m, the average access time is 1 + 100m. Setting this time to 1.5 and solving for m requires a cache miss rate of 0.5%.

Gene Amdahl, 1922 -

- **Amdahl's Law:** the effort spent increasing the performance of a subsystem is wasted unless the subsystem affects a large percentage of overall performance
- Co-founded 3 companies, including one called Amdahl Corporation in 1970

CACHES

- Cache memory is a small and high-speed memory unit placed between the CPU and the main memory.
- It stores frequently accessed instructions and data so that the processor can access them quickly.
- Cache memory reduces the average memory access time and improves overall system performance.
- Cache is implemented using SRAM technology, which is faster than DRAM used in main memory.


Processor Chip
CLK
CPU
Cache
Main
Memory
Hard
Disk

Need for Cache Memory

Modern processors operate at very high speeds, whereas main memory is relatively slow.

This difference creates a performance gap between the CPU and memory.

Problems without Cache:

- CPU remains idle while waiting for data
- Increased execution time
- Poor system performance

Solution:

Cache memory stores frequently used data closer to the processor, reducing memory access delay.

Cache in Memory Hierarchy

Cache memory acts as an intermediate layer between CPU and main memory.

Working:

- When the processor attempts to access data, it first checks the cache for the data.
- If the cache hits, the data is available immediately.
- If the cache misses, the processor fetches the data from main memory and places it in the cache for future use.
- To accommodate the new data, the cache must *replace* old data.

Cache Hit and Cache Miss

A. Cache Hit

A cache hit occurs when the requested data is found in cache memory.

Advantages:

- Faster data access
- Reduced memory latency
- Improved CPU performance

B. Cache Miss

A cache miss occurs when the requested data is not found in cache memory.

Effects:

- Data must be fetched from main memory
- Increased access time
- Reduced system performance

Cache Terminology

1. Cache Capacity (C)
   - Total amount of data that cache can store
   - Measured in KB or MB
2. Block Size (b)
   - Amount of data transferred between memory and cache at one time
   - Larger block size increases spatial locality
3. Number of Blocks (B)
   - Total blocks present in cache

     Formula: $B = C / b$
4. Associativity (N)
   - Number of blocks in each set
   - Determines flexibility of data placement
5. Number of Sets (S)
   - Total number of sets in cache
   - Each memory address maps to exactly one cache set

     Formula: $S = B / N$

Cache design questions

1. What data is held in the cache?
2. How is data found? and
3. What data is replaced to make room for new data when the cache is full?

What Data is Held in the Cache?

The cache must guess what data will be needed based on the past pattern of memory accesses.

The cache exploits temporal and spatial locality to achieve a low miss rate.

Temporal locality means that the processor is likely to access a piece of data again soon if it has accessed that data recently.

Therefore, when the processor loads or stores data that is not in the cache, the data is copied from main memory into the cache.

Subsequent requests for that data hit in the cache.

Spatial locality means that, when the processor accesses a piece of data, it is also likely to access data in nearby memory locations.

Therefore, when the cache fetches one word from memory, it may also fetch several adjacent words.

This group of words is called a *cache block* or *cache line*.

The number of words in the cache block, b, is called the *block size*.

A cache of capacity C contains $B = C / b$ blocks.

How is Data Found in Cache?

A cache is organized into S sets, each of which holds one or more blocks of data.

The relationship between the address of data in main memory and the location of that data in the cache is called the **mapping**.

Each memory address maps to exactly one set in the cache.

Some of the address bits are used to determine which cache set contains the data.

If the set contains more than one block, the data may be kept in any of the blocks in the set.

Caches are categorized based on the number of blocks in a set.

Caches are classified into three types:

1. Direct Mapped Cache

   In this type, each set contains one block.

   Cache has S = B sets

   A particular main memory address maps to a unique block in the cache. It is simple and fast but suffers from high conflict misses.

2. N-Way Set Associative Cache

   In this organization, each set contains N blocks.

   The address maps to a unique set

   No of sets (S) = B/N sets.

   But the data from that address can go in any of the N blocks in that set.

3. Fully Associative Cache

   A *fully associative* cache has only 1 set. (S = 1) All cache blocks belong to one set.

   A memory block can be placed anywhere in the cache.

   Data can go in any of the *B* blocks in the set. Hence, a fully associative cache is another name for a *B*-way set associative cache.

   It offers maximum flexibility and minimum conflict misses but requires complex hardware.

Consider a RISC-V memory system with 32-bit addresses and 32-bit words.

The memory is byte-addressable, and each word is four bytes, so the memory consists of 2^{30} words aligned on word boundaries.

Cache hash an eight word capacity (*C*) for the sake of simplicity.

Block size (b) is one-word

Direct Mapped Cache

A *direct mapped* cache has one block in each set, so it is organized into $S = B$ sets.

To understand the mapping of memory addresses onto cache blocks, imagine main memory as being mapped into b-word blocks, just as the cache is.

An address in block 0 of main memory maps to set 0 of the cache. An address in block 1 of main memory maps to set 1 of the cache, and so forth until an address in block $B - 1$ of main memory maps to block $B - 1$ of the cache.

There are no more blocks of the cache, so the mapping wraps around, such that block B of main memory maps to block 0 of the cache.

For eg. Consider a direct mapped cache with a capacity of eight words and a block size of one word shown in figure 8.5.

The cache has eight sets, each of which contains a one-word block.

The data at addresses 0x00000004, 0x00000024, …, 0xFFFFFFE4 all map to set 1, as shown in blue.

Likewise, data at addresses 0x00000010, …, 0xFFFFFFF0 all map to set 4, and so forth.

Each main memory address maps to exactly one set in the cache.

The two least significant bits of the 32-bit address are called the *byte* ***offset*** because they indicate the byte within the word.

The next three bits are called the ***set*** *bits* because they indicate the set to which the address maps.

(In general, the number of set bits is $\log_2 S$.)

The remaining 27 **tag** bits indicate the memory address of the data stored in a given cache set

Capacity (***C*** = 8 words)
Block size (***b*** = 1 word)
So, number of blocks (***B*** = 8)

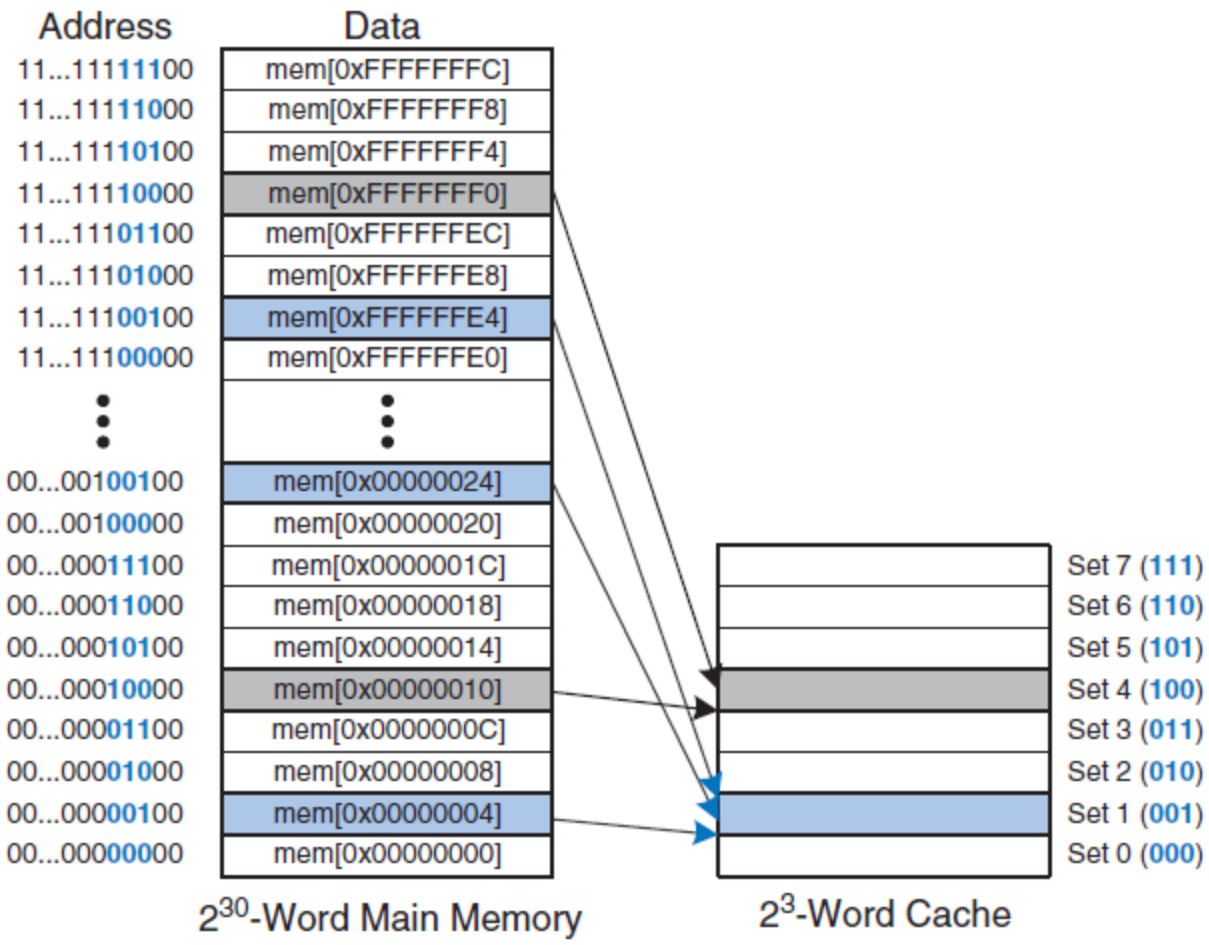


Figure 8.5. Mapping of main memory to a direct mapped cache

Figure 8.6 shows the cache fields for address 0xFFFFFFE4. It maps to set 1 and its tag is all l's.

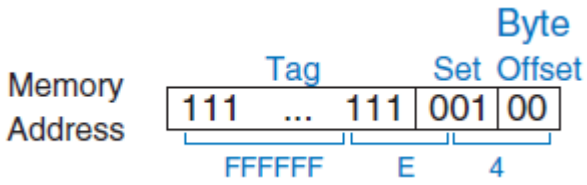


Figure 8.6 Cache fields for address 0xFFFFFFE4 when mapping to the cache in Figure 8.5

Q) To what cache set in Figure 8.5 does the word at address 0x00000014 map? Name another address that maps to the same set.

Solution

The two least significant bits of the address are 00, because the address is word aligned.

The next three bits are 101, so the word maps to set 5.

Words at addresses 0x34, 0x54, 0x74, …, 0xFFFFFFF4 all map to this same set.

Q) Find the number of set and tag bits for a direct mapped cache with 1024 (2^{10}) sets and a one-word block size. The address size is 32 bits.

Solution

A cache with 2^{10} sets requires $\log_2(2^{10}) = 10$ set bits.

The two least significant bits of the address are the byte offset, and the remaining $32 - 10 - 2 = 20$ bits form the tag.

Sometimes, such as when the computer first starts up, the cache sets contain no data at all.

The cache uses a *valid bit* for each set to indicate whether the set holds meaningful data.

If the valid bit is 0, the contents are meaningless.

Figure 8.7 shows the hardware for the direct mapped cache of Figure 8.5.

The cache is constructed as an eight-entry SRAM. Each entry, or set, contains one line consisting of 32 bits of data, 27 bits of tag, and 1 valid bit.

The cache is accessed using the 32-bit address.

The two least significant bits, the byte offset bits, are ignored for word accesses.

The next three bits, the set bits, specify the entry or set in the cache.

A load instruction reads the specified entry from the cache and checks the tag and valid bits. If the tag matches the most significant 27 bits of the requested address and the valid bit is 1, the cache hits and the data is returned to the processor.

Otherwise, the cache misses and the memory system must fetch the data from main memory.

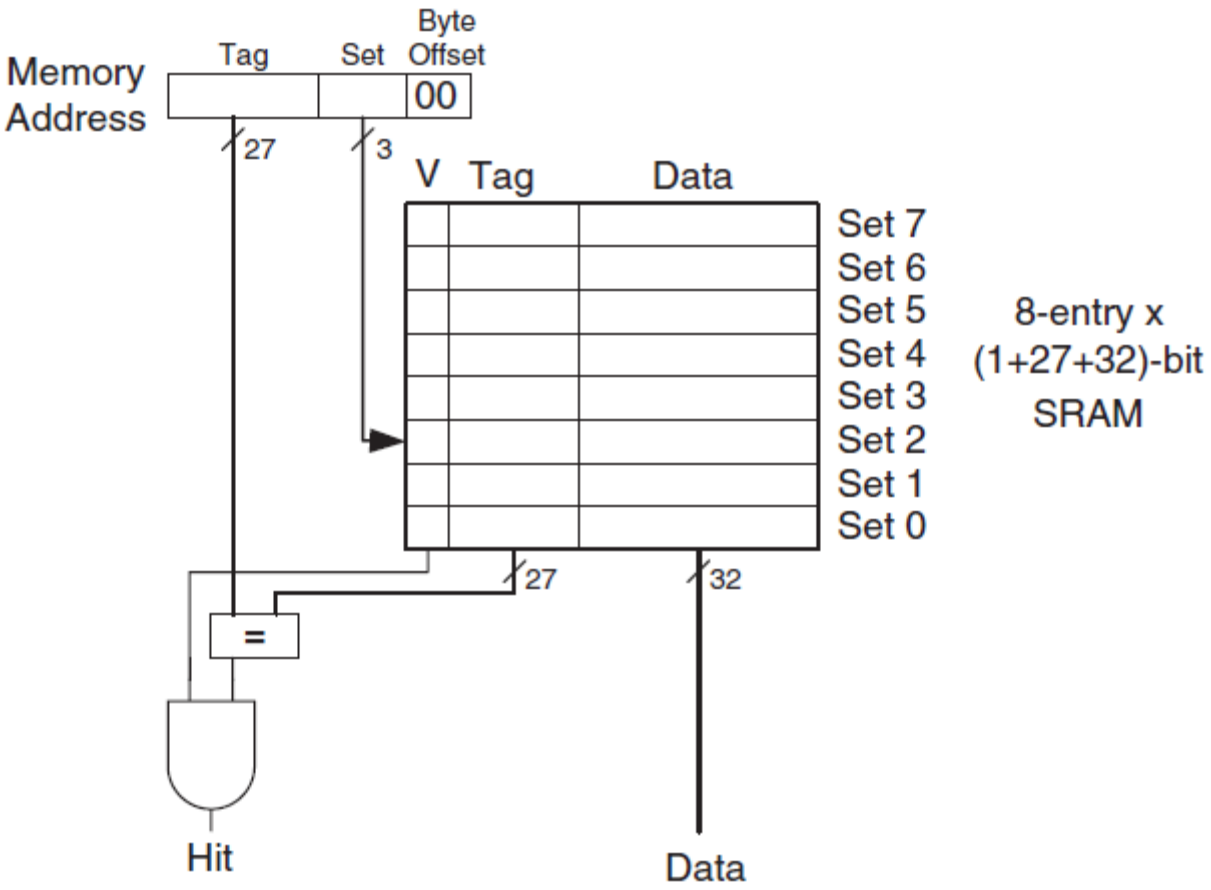


Figure 8.7 Direct mapped cache with 8 sets

Example 8.6 TEMPORAL LOCALITY WITH A DIRECT MAPPED CACHE

Loops are a common source of temporal and spatial locality in applications.

Q) Using the eight-entry cache of Figure 8.7, show the contents of the cache after executing the following silly loop in RISC-V assembly code. Assume that the cache is initially empty. What is the miss rate?

```
        addi s0, zero, 5
        addi s1, zero, 0
LOOP:   beq  s0, zero, DONE
        lw   s2, 4(s1)
        lw   s3, 12(s1)
        lw   s4, 8(s1)
        addi s0, s0, -1
        j    LOOP
DONE:
```

Solution

The program contains a loop that repeats for five iterations.

Each iteration involves three memory accesses (loads), resulting in 15 total memory accesses.

The first time the loop executes, the cache is empty and the data must be fetched from main memory locations 0x4, 0xC, and 0x8 into cache sets 1, 3, and 2, respectively.

However, the next four times the loop executes, the data is found in the cache.

Figure 8.8 shows the contents of the cache during the last request to memory address 0x4.

The tags are all 0 because the upper 27 bits of the addresses are 0.

The miss rate is 3/15 = 20%.

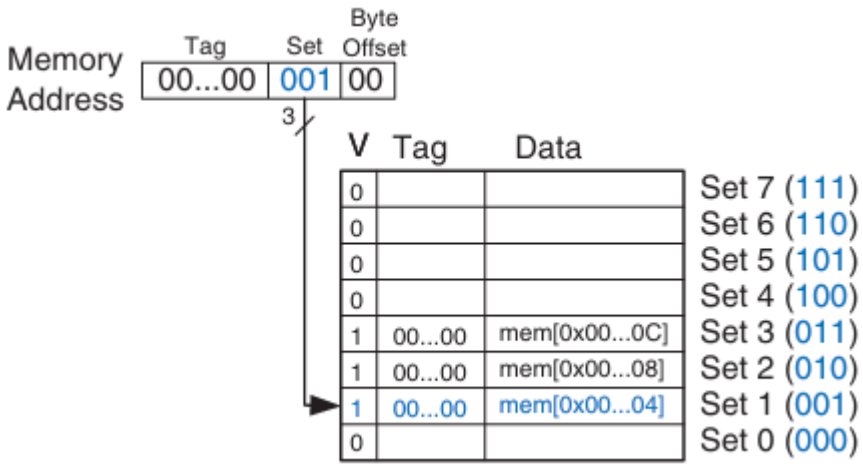


Figure 8.8 Direct mapped cache contents

When two recently accessed addresses map to the same cache block, a conflict occurs, and the most recently accessed address evicts the previous one from the block.

Direct mapped caches have only one block in each set, so two addresses that map to the same set always cause a conflict. Example 8.7 illustrates conflicts.

Example 8.7 CACHE BLOCK CONFLICT

Q) What is the miss rate when the following loop is executed on the eight-word direct mapped cache from Figure 8.7? Assume that the cache is initially empty.

```
        addi s0, zero, 5
        addi s1, zero, 0
LOOP:   beq  s0, zero, DONE
        lw   s2, 0x4(s1)
        lw   s4, 0x24(s1)
        addi s0, s0, -1
        j    LOOP

DONE:
```

Solution

Memory addresses 0x4 and 0x24 both map to set 1.

During the initial execution of the loop, data at address 0x4 is loaded into set 1 of the cache.

Then, data at address 0x24 is loaded into set 1, evicting the data from address 0x4.

Upon the second execution of the loop, the pattern repeats and the cache must refetch data at address 0x4, evicting data from address 0x24.

The two addresses conflict and the miss rate is 100%

Multiway Set Associative Cache

An N-way set associative cache reduces conflicts by providing N blocks in each set where data mapping to that set might be found.

Each memory address still maps to a specific set, but it can map to any one of the N blocks in the set.

Hence, a direct mapped cache is another name for a one-way set associative cache.

N is also called the degree of associativity of the cache.

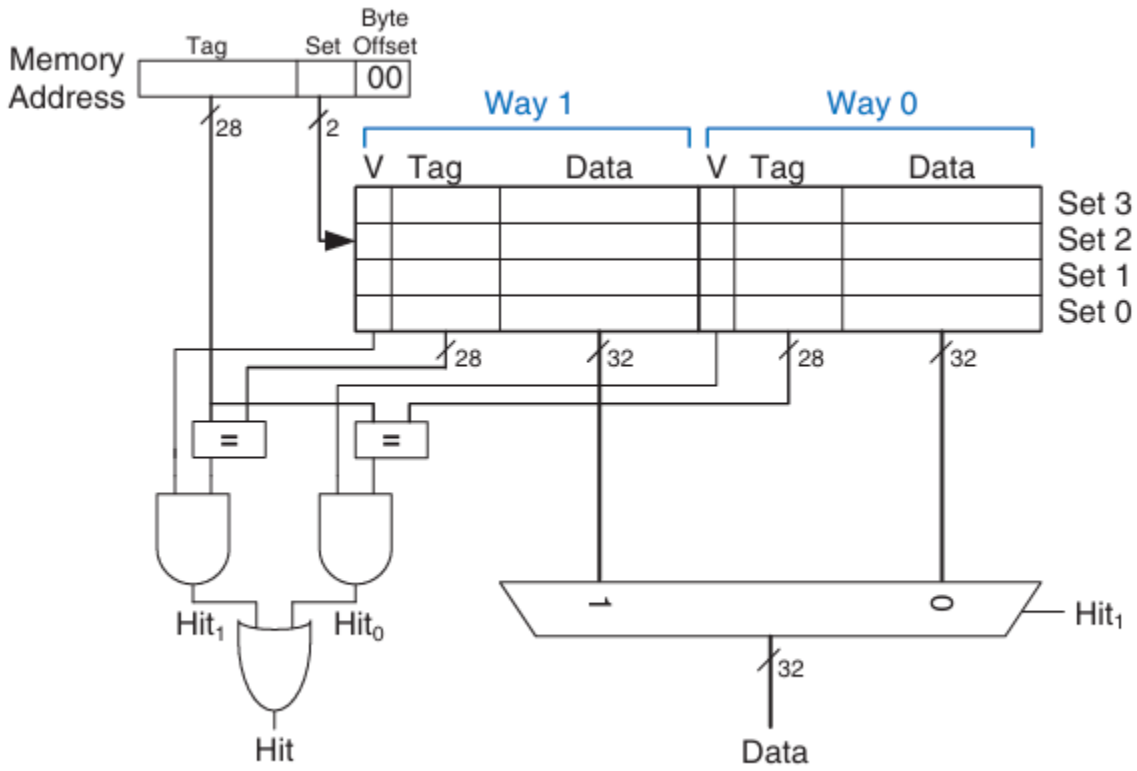


Figure 8.9 Two-way set associative cache

Figure 8.9 shows the hardware for a C = 8-word, N = 2-way set associative cache.

The cache now has only S = 4 sets rather than 8.

Thus, only $\log_2 4 = 2$ set bits rather than 3 are used to select the set.

The tag increases from 27 to 28 bits.

Each set contains two ways or degrees of associativity.

Each way consists of a data block and the valid and tag bits.

The cache reads blocks from both ways in the selected set and checks the tags and valid bits for a hit.

If a hit occurs in one of the ways, a multiplexer selects data from that way.

Set associative caches generally have lower miss rates than direct mapped caches of the same capacity because they have fewer conflicts.

However, set associative caches are usually slower and somewhat more expensive to build because of the output multiplexer and additional comparators.

They also raise the question of which way to replace when both ways are full; this is addressed further in Section 8.3.3.

Most commercial systems use set associative caches.

Example 8.8 SET ASSOCIATIVE CACHE MISS RATE

Q) Repeat Example 8.7 using the eight-word two-way set associative cache from Figure 8.9.

```
        addi s0, zero, 5
        addi s1, zero, 0
LOOP:   beq  s0, zero, DONE
        lw   s2, 0x4(s1)
        lw   s4, 0x24(s1)
        addi s0, s0, -1
        j    LOOP

DONE:
```

Solution

Both memory accesses, to addresses 0x4 and 0x24, map to set 1.

However, the cache has two ways, so it can accommodate data from both addresses.

During the first loop iteration, the empty cache misses both addresses and loads both words of data into the two ways of set 1, as shown in Figure 8.10.

On the next four iterations, the cache hits.

Hence, the miss rate is 2/10 = 20%.

Recall that the direct mapped cache of the same size from Example 8.7 had a miss rate of 100%.

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...00	mem[0x00...24]	1	00...10	mem[0x00...04]	Set 1
0			0			Set 0

Figure 8.10 Two-way set associative cache contents

Fully Associative Cache

A fully associative cache contains a single set with B ways, where B is the number of blocks.

A memory address can map to a block in any of these ways.

A fully associative cache is another name for a B-way set associative cache with one set.

Figure 8.11 shows the SRAM array of a fully associative cache with eight blocks.

Upon a data request, eight tag comparisons (not shown) must be made because the data could be in any block.

Similarly, an 8:1 multiplexer chooses the proper data if a hit occurs.

Fully associative caches tend to have the fewest conflict misses for a given cache capacity, but they require more hardware for additional tag comparisons.

They are best suited to relatively small caches because of the large number of comparators.

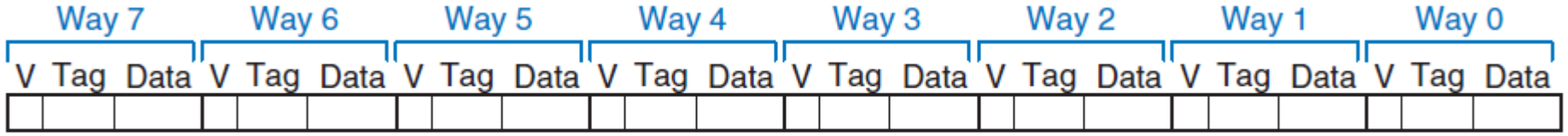


Figure 8.11 Eight-block fully associative cache

Block Size

The previous examples were able to take advantage only of temporal locality because the block size was one word.

To exploit spatial locality, a cache uses larger blocks to hold several consecutive words.

The advantage of a block size greater than one is that when a miss occurs and the word is fetched into the cache, the adjacent words in the block are also fetched. Therefore, subsequent accesses are more likely to hit because of spatial locality.

However, a large block size means that a fixed-size cache will have fewer blocks.

This may lead to more conflicts, increasing the miss rate.

Moreover, it takes more time to fetch the missing cache block after a miss because more than one data word is fetched from main memory.

The time required to load the missing block into the cache is called the *miss penalty*.

If the adjacent words in the block are not accessed later, the effort of fetching them is wasted.

Nevertheless, most real programs benefit from larger block sizes.

Figure 8.12 shows the hardware for a C = 8-word direct mapped cache with a b = 4-word block size.

The cache now has only $B = C/b = 2$ blocks.

A direct mapped cache has one block in each set, so this cache is organized as two sets.

Thus, only $\log_2 2 = 1$ bit is used to select the set.

A multiplexer is now needed to select the word within the block.

The multiplexer is controlled by the $\log_2 4 = 2$ *block offset bits* of the address. The most significant 27 address bits form the tag.

Only one tag is needed for the entire block, because the words in the block are at consecutive addresses.

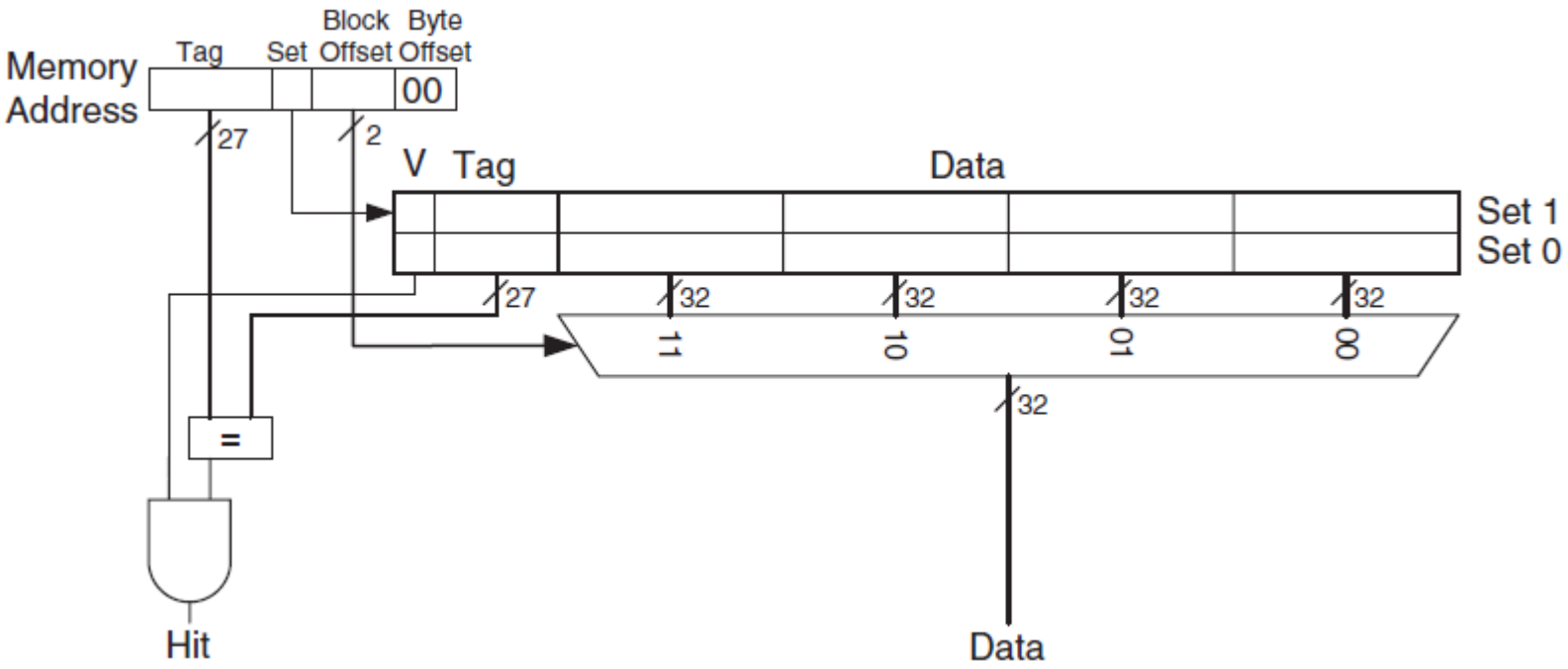


Figure 8.12 Direct mapped cache with two sets and a four-word block size

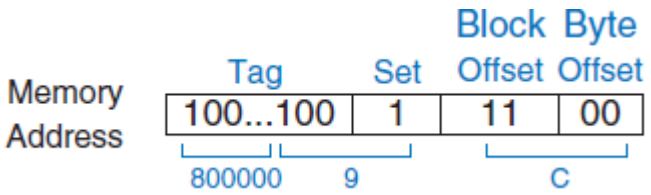


Figure 8.13 Cache fields for address 0x8000009C when mapping to the cache of Fig. 8.12

Figure 8.13 shows the cache fields for address 0x8000009C when it maps to the direct mapped cache of Figure 8.12.

The byte offset bits are always 0 for word accesses.

The next $\log_2 b = 2$ block offset bits indicate the word within the block and the next bit indicates the set.

The remaining 27 bits are the tag.

Therefore, word 0x8000009C maps to set 1, word 3 in the cache.

The principle of using larger block sizes to exploit spatial locality also applies to associative caches.

Example 8.9 SPATIAL LOCALITY WITH A DIRECT MAPPED CACHE

Q) Show the contents of the cache after executing the following loop in RISC-V assembly code for the eight-word direct mapped cache with a four-word block size. The word size is 4 bytes. Assume that the cache is initially empty. What is the miss rate?

```
        addi s0, zero, 5
        addi s1, zero, 0
LOOP:   beq  s0, zero, DONE
        lw   s2, 4(s1)
        lw   s3, 12(s1)
        lw   s4, 8(s1)
        addi s0, s0, -1
        j    LOOP
DONE:
```

Solution

Figure 8.14 shows the contents of the cache after the first memory access.

On the first loop iteration, the cache misses on the access to memory address 0x4.

This access loads data at addresses 0x0 through 0xC into the cache block.

All subsequent accesses (as shown for address 0xC) hit in the cache.

Hence, the miss rate is 1/15 = 6.67%.

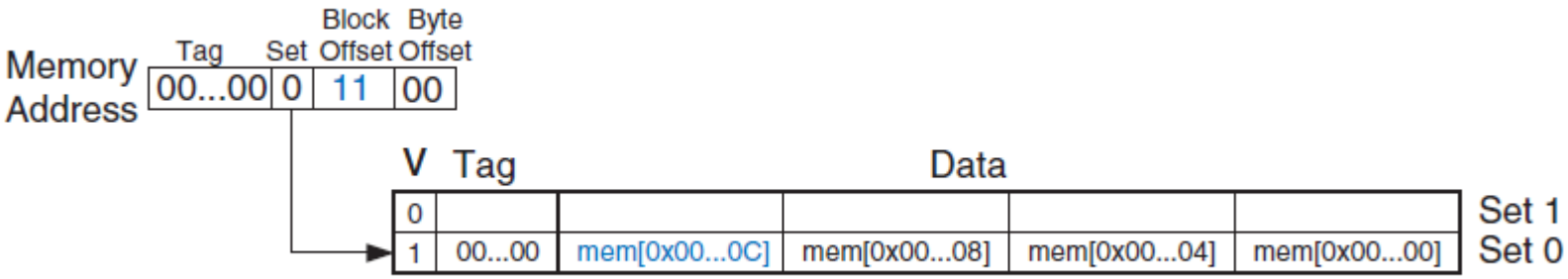


Figure 8.14 Cache contents with a block size *b* of four words

Putting it All Together

Caches are organized as two-dimensional arrays. The rows are called *sets*, and the columns are called *ways*.

Each entry in the array consists of a data block and its associated valid and tag bits.

Caches are characterized by

- capacity C
- block size b (and number of blocks, $B = C/b$)
- number of blocks in a set (N)

Table 8.2 Cache organizations

Organization	Number of Ways (N)	Number of Sets (S)
Direct Mapped	1	B
Set Associative	$1 < N < B$	B/N
Fully Associative	B	1

Table 8.2 summarizes the various cache organizations.

Each address in memory maps to only one set but can be stored in any of the ways.

Cache capacity, associativity, set size, and block size are typically powers of two.

This makes the cache fields (tag, set, and block offset bits) subsets of the address bits.

Increasing the associativity *N* usually reduces the miss rate caused by conflicts.

But higher associativity requires more tag comparators.

Increasing the block size *b* takes advantage of spatial locality to reduce the miss rate.

However, it decreases the number of sets in a fixed sized cache and, therefore, could lead to more conflicts.

It also increases the miss penalty.

Cache Replacement

What Data is Replaced?

In a direct mapped cache, each address maps to a unique block and set.

If a set is full when new data must be loaded, the block in that set is replaced with the new data.

In set associative and fully associative caches, the cache must choose which block to evict when a cache set is full.

The principle of temporal locality suggests that the best choice is to evict the least recently used block because it is least likely to be used again soon.

Hence, most associative caches have a *least recently used* (*LRU*) replacement policy.

In a two-way set associative cache, a *use bit*, *U*, indicates which way within a set was least recently used.

Each time one of the ways is used, *U* is adjusted to indicate the other way.

For set associative caches with more than two ways, tracking the least recently used way becomes complicated.

To simplify the problem, the ways are often divided into two groups and *U* indicates which *group* of ways was least recently used.

Upon replacement, the new block replaces a random block within the least recently used group.

Such a policy is called *pseudo-LRU* and is good enough in practice.

Cache Replacement

In a direct mapped cache, each address maps to only one block. If the block is full, it is automatically replaced. No replacement decision is needed in direct mapped cache.

In set associative and fully associative caches, a set has multiple blocks. When the set is full, the cache must choose which block to replace. The most common replacement policy is **Least Recently Used (LRU)**.

Least Recently Used (LRU)

- LRU replaces the block that has not been used for the longest time.
- LRU is based on temporal locality.
- In a 2-way set associative cache, one use bit (U) is used.
- The use bit shows which way is least recently used.
- When one way is accessed, the other way becomes LRU.
- In caches with more than two ways, exact LRU is difficult to implement.
- A simplified method called pseudo-LRU is used.
- In pseudo-LRU, ways are divided into groups.
- A block from the least recently used group is replaced.
- LRU reduces unnecessary replacements and improves performance

Example 8.10 LRU REPLACEMENT

Show the contents of an eight-word two-way set associative cache after executing the following code. Assume LRU replacement, a block size of one word, and an initially empty cache.

```
addi t0, zero, 0
lw   s1, 0x4(t0)
lw   s2, 0x24(t0)
lw   s3, 0x54(t0)
```

Solution

The first two instructions load data from memory addresses 0x4 and 0x24 into set 1 of the cache, shown in Figure 8.15(a).

U = 0 indicates that data in way 0 was the least recently used.

The next memory access, to address 0x54, also maps to set 1 and replaces the least recently used data in way 0, as shown in Figure 8.15(b).

The use bit *U* is set to 1 to indicate that data in way 1 was the least recently used.

	Way 1			Way 0			
	V	U	Tag	Data	V	Tag	Data	
	0	0			0			Set 3 (11)
	0	0			0			Set 2 (10)
	1	0	00...010	mem[0x00...24]	1	00...000	mem[0x00...04]	Set 1 (01)
	0	0			0			Set 0 (00)

(a)

	Way 1			Way 0			
	V	U	Tag	Data	V	Tag	Data	
	0	0			0			Set 3 (11)
	0	0			0			Set 2 (10)
	1	1	00...010	mem[0x00...24]	1	00...101	mem[0x00...54]	Set 1 (01)
	0	0			0			Set 0 (00)

(b)

Figure 8.15 Two-way associative cache with LRU replacement

Advanced Cache Design

Modern systems use multiple levels of caches to decrease memory access time.

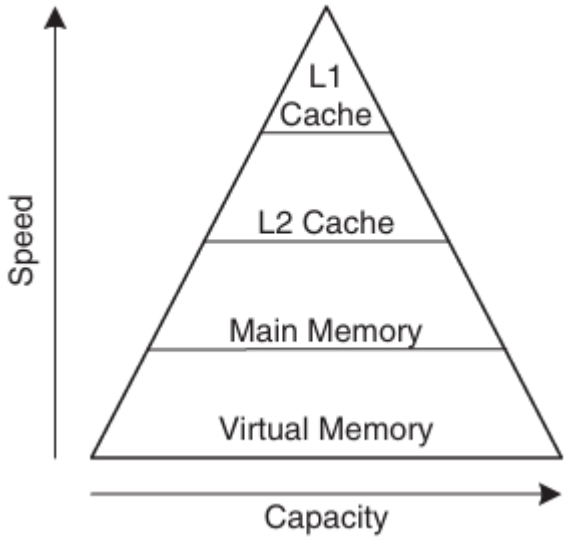


Figure 8.16 Memory hierarchy with two levels of cache

Multiple-Level Caches

Large caches are beneficial because they are more likely to hold data of interest and, therefore, have lower miss rates.

However, large caches tend to be slower than small ones.

Modern systems often use at least two levels of caches, as shown in Figure 8.16.

The first-level (L1) cache is small enough to provide a one- or two-cycle access time.

The second-level (L2) cache is also built from SRAM but is larger—and, therefore, slower—than the L1 cache.

The processor first looks for the data in the L1 cache.

If the L1 cache misses, the processor looks in the L2 cache.

If the L2 cache misses, the processor fetches the data from main memory.

Many modern systems add even more levels of cache to the memory hierarchy because accessing main memory is so slow.

Example 8.11 SYSTEM WITH AN L2 CACHE

Use the system of Figure 8.16 with access times of 1, 10, and 100 cycles for the L1 cache, L2 cache, and main memory, respectively. Assume that the L1 and L2 caches have miss rates of 5% and 20%, respectively. Specifically, of the 5% of accesses that miss the L1 cache, 20% of those also miss the L2 cache. What is the average memory access time (AMAT)?

Solution

Each memory access checks the L1 cache. When the L1 cache misses (5% of the time), the processor checks the L2 cache. When the L2 cache misses (20% of the time), the processor fetches the data from main memory. Using Equation 8.2, we calculate the average memory access time as follows: 1 cycle + 0.05[10 cycles + 0.2(100 cycles)] = 2.5 cycles.

Reducing Miss Rate

Cache misses can be reduced by changing capacity, block size, and/or associativity.

The first step to reducing the miss rate is to understand the causes of the misses.

The misses can be classified as **compulsory, capacity, and conflict**.

The first request to a cache block is called a *compulsory miss*, because the block must be read from memory regardless of the cache design.

Capacity misses occur when the cache is too small to hold all concurrently used data.

Conflict misses are caused when several addresses map to the same set and evict blocks that are still needed.

Changing cache parameters can affect one or more types of cache miss.

For example, **increasing cache capacity** can **reduce conflict and capacity misses**, but it **does not affect compulsory misses**.

On the other hand, **increasing block size** could **reduce compulsory misses** (due to spatial locality) but might actually *increase* conflict misses (because more addresses would map to the same set and could conflict).

Memory systems are complicated enough that the best way to evaluate their performance is by running benchmarks while varying cache parameters.

Figure 8.17 plots miss rate versus cache size and degree of associativity for the SPEC2000 benchmark.

This benchmark has a small number of compulsory misses, shown by the dark region near the x-axis.

As expected, when cache size increases, capacity misses decrease.

Increased associativity, especially for small caches, **decreases the number of conflict** misses shown along the top of the curve.

Increasing associativity beyond four or eight ways provides only small decreases in miss rate.

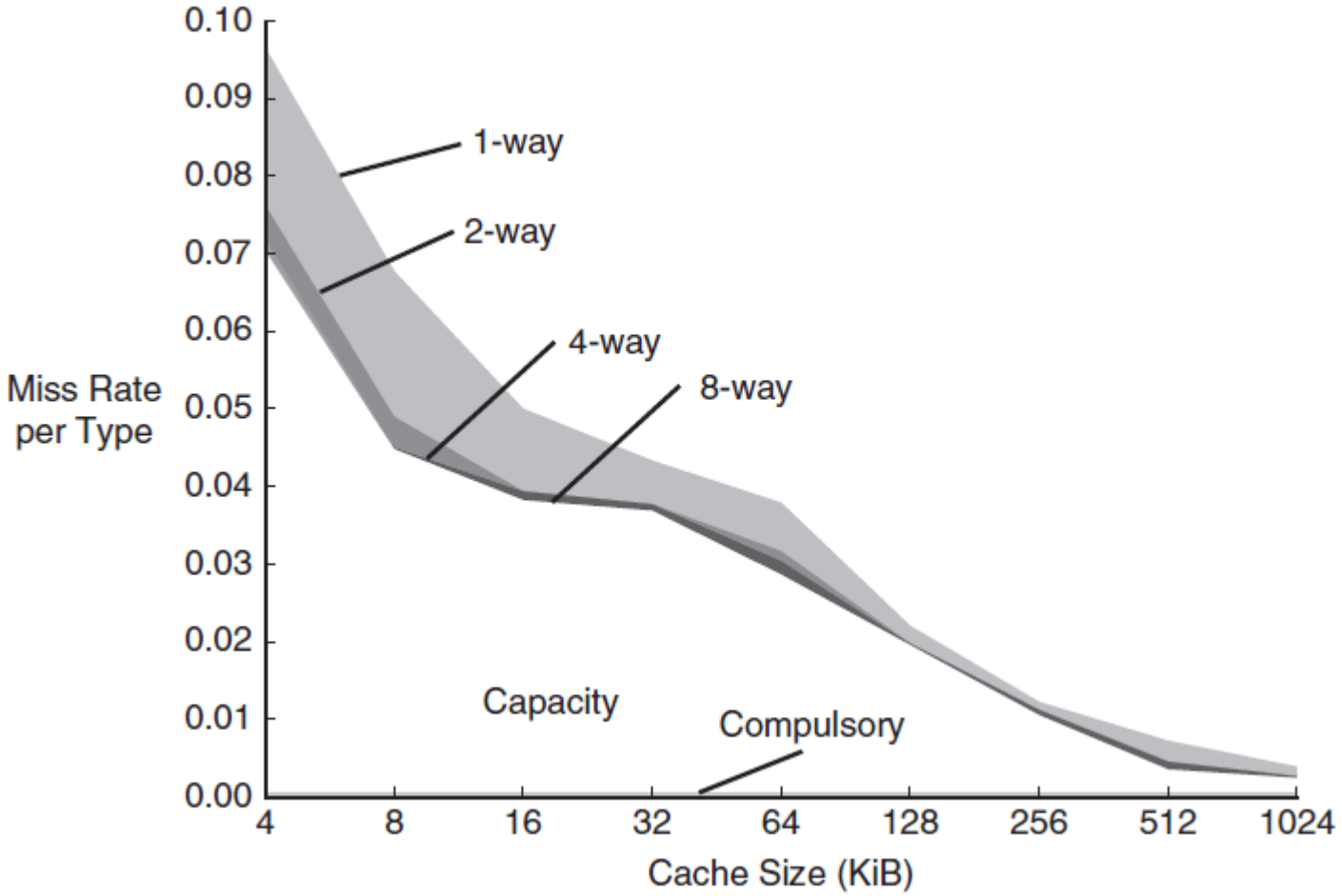


Figure 8.17 Miss rate versus cache size and associativity on SPEC2000 benchmark Adapted with permission from Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed., Morgan Kaufmann, 2012

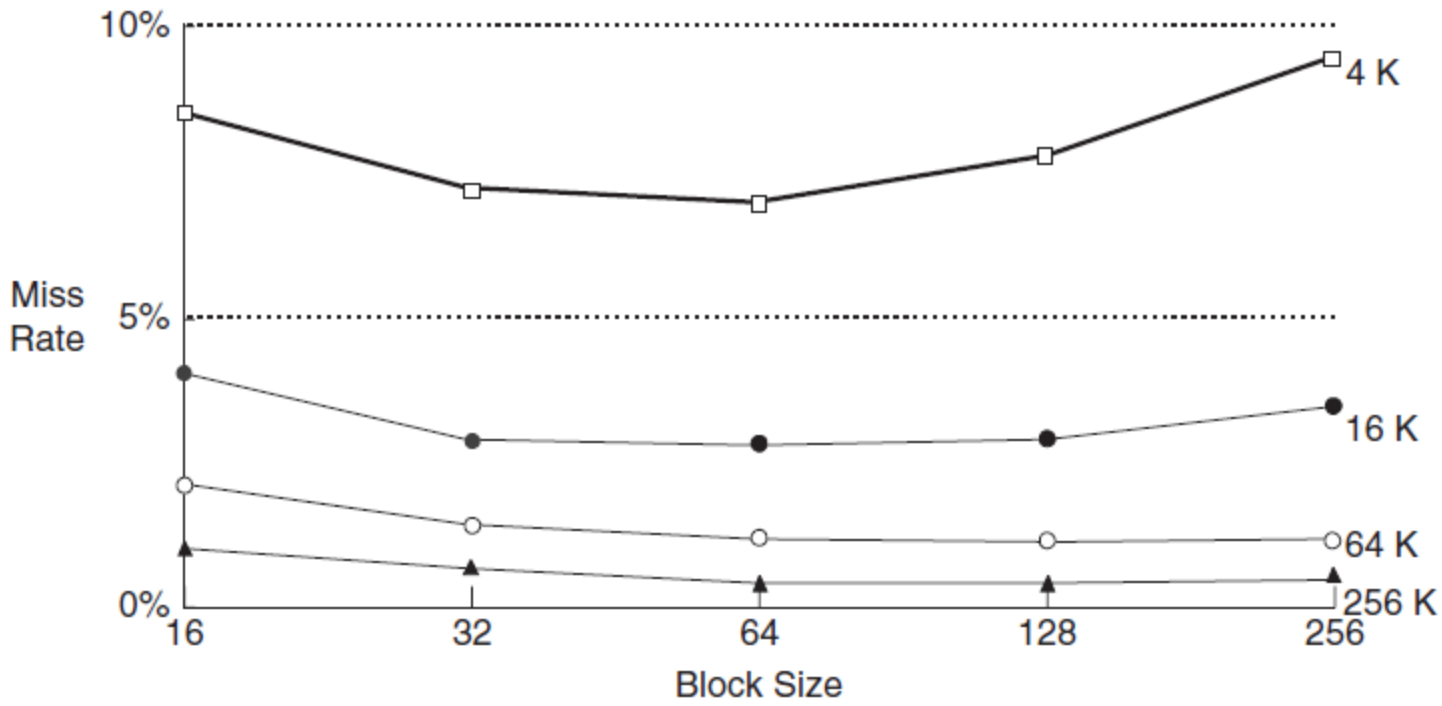


Figure 8.18 Miss rate versus block size and cache size on SPEC92 benchmark
Adapted with permission from Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed., Morgan Kaufmann, 2012

As mentioned, miss rate can also be decreased by using larger block sizes that take advantage of spatial locality.

But as block size increases, the number of sets in a fixed-size cache decreases, increasing the probability of conflicts.

Figure 8.18 plots miss rate versus block size (in number of bytes) for caches of varying capacity.

For small caches, such as the 4 KB cache, increasing the block size beyond 64 bytes *increases* the miss rate because of conflicts.

For larger caches, increasing the block size beyond 64 bytes does not change the miss rate.

However, large block sizes might still increase execution time because of the larger miss penalty, the time required to fetch the missing cache block from main memory.

Write Policy

The previous sections focused on memory loads.

Memory stores, or writes, follow a similar procedure as loads.

Upon a memory store, the processor checks the cache.

If the cache misses, the cache block is fetched from main memory into the cache, and then the appropriate word in the cache block is written.

If the cache hits, the word is simply written to the cache block.

Caches are classified as either **write-through** or **write-back**.

In a ***writethrough*** cache, the data written to a cache block is simultaneously written to main memory.

In a *write-back* cache, a ***dirty bit*** (*D*) is associated with each cache block.

D is 1 when the cache block has been written and 0 otherwise.

Dirty cache blocks are written back to main memory only when they are evicted from the cache.

A write-through cache requires no dirty bit but usually requires more main memory writes than a write-back cache.

Modern caches are usually write-back because main memory access time is so large.

Example 8.12 WRITE-THROUGH VERSUS WRITE-BACK

Suppose a cache has a block size of four words. How many main memory accesses are required by the following code when using each write policy: write-through or write-back?

```
addi t5, zero, 0
sw   t1, 0(t5)
sw   t2, 12(t5)
sw   t3, 8(t5)
sw   t4, 4(t5)
```

Solution

All four store instructions write to the same cache block.

With a writethrough cache, each store instruction writes a word to main memory, requiring four main memory writes.

A write-back policy requires only one main memory access, when the dirty cache block is evicted.